ChatGPT

House of Wolves 2013 Flash RTS Analytical Porting Report

Executive summary

House of Wolves is a 2D medieval real-time strategy Flash game by Louissi/Loussi, originally published on Armor Games in May 2013. The most stable cross-source description is consistent: the player begins with a single settler, repairs an initial guard/watch tower, builds a hut, gathers resources, trains troops, pushes east against Lord Vilereck, and simultaneously survives pressure from hostile creatures on the western side of the map. Official and semi-official descriptions consistently frame it as a side-scrolling RTS tribute to *Age of Empires* and *StarCraft*, centered on harvesting, hunting, building, rescue, and conquest. One source discrepancy is worth noting up front: Armor Games lists the game on May 8, 2013, while the community wiki states May 13, 2013. That is best read as a publication-versus-release-page discrepancy rather than a material contradiction. [1]

A faithful Pygame port is technically feasible. The original game's content scope is modest by RTS standards: a continuous 2D side-scrolling map, one player faction with nine unit types, five economy resources, roughly ten player building types, a handful of enemy/neutral factions, and a mostly local-input, single-player simulation loop. The strongest implementation recommendation is **not** a full tile-grid RTS with expensive pathfinding, but a **continuous side-scroller with walkable ground bands, static footprint obstacles, lightweight local steering, optional coarse detours around building clusters, and command queues modeled per unit**. That architecture best matches what the source material appears to be mechanically and visually, while preserving headroom for stronger controls such as Shift-queueing, control groups, and improved AI. [2]

The source record also shows where the original design was weak. Multiple community analyses observed that massing Settlers or Spearmen could trivialize the game, because renewable wood/food economies scale quickly, advanced resources are spatially limited, and at least one design analysis reports that unit creation is effectively instantaneous rather than timer-gated. For a portfolio port that starts faithful and then evolves, the best course is to recreate the observed behavior first, but to implement the simulation cleanly enough that later balancing changes—production times, smarter caps, resource depletion tuning, and more differentiated counters—can be added without rewriting core systems. [3]

From an engineering standpoint, a solo developer should expect something in the rough range of **180 to 320 hours** for a polished, faithful single-player desktop port with original replacement art, a complete core roster, upgrades, basic save/load, and solid performance work. A narrower "faithful playable first version" can land earlier, but the original's apparent simplicity is deceptive: selection, command routing, gather/deposit loops, placement validation, ranged and melee combat, rescue states, wave scheduling, and robust UI feedback are where most of the implementation time will actually go. Pygame's `Sprite` / `Group`, `Rect`, timing, and surface APIs are sufficient for this scale of project when used carefully. [4]

Source profile and observed game structure

The official and near-official source set establishes the game's identity and high-level structure clearly. Armor Games describes *House of Wolves* as a tribute to RTS classics where the player "harvest[s] trees, hunt[s] animals, explore[s], build[s] and conquer[s]" while pursuing vengeance on Lord Vilereck; Newgrounds repeats the same framing and specifically mentions rescuing princesses, training wizards, and building fortresses to recruit knights. The community wiki adds that story mode starts in the West Frontier and that the tutorial begins immediately upon arrival. [5]

Release-era guides and community records are especially valuable because they preserve concrete UI and input details that are not exposed on the current Armor Games page. A May 9, 2013 guide on PTT describes the tutorial order in detail: move a worker, repair the watch tower, build a hut, chop wood, train five settlers, gather food by killing a nearby deer, kill three dark spiders to the west, then build either a Barracks or Archery building. That same guide records drag-box selection, double-click to select same-type units, Shift-additive selection, Ctrl+number control groups, and right-click to perform actions. It also describes the HUD: a skull icon in the upper-right indicating the timer until the next attack wave; a resource display showing wood, stone, iron, gold, food, and population; and a lower-right worker-status panel that can be clicked to move the camera. [6]

Independent contemporaneous writeups fill in the larger strategic structure. StratBuster describes the player start as a watch tower plus one settler in the wilderness, with the main objective being to march east against Lord Vilereck while fending off an endless stream of dark creatures from the west. A May 2013 review/guide also describes the game as positionally split: Vilereck lies to the right, hostile wilderness and monsters to the left, and strong play often requires simultaneously stabilizing both fronts. [7]

The indexed video record is sufficient to confirm that gameplay videos exist and were widely circulated, but the accessible search snippets do not expose internal minute-second captions reliably. What can be confirmed from indexed results is that there are at least a hard-difficulty full walkthrough listed at **16:45**, an easy-difficulty walkthrough at **12:35**, and a separate gameplay video whose snippet explicitly mentions RTS-style grouping into "quick access clusters." Because the accessible web results do not surface full captions or transcript timings, the evidence table below combines those video references with release-era screenshot/text examples rather than pretending to recover timecodes that were not accessible from indexed snippets. [8]

Evidence artifact	What it confirms	Source
Armor Games game page	RTS tribute framing; harvesting, hunting, building, conquering; vengeance against Lord Vilereck	[9]
Community wiki game page	Tutorial starts on arrival in the West Frontier; one-settler opening via Poki blurb; repair guard tower → build hut	[10]
Gameflare and Miniplay wrappers	Mouse input; right-click action; third-party embeds commonly frame the game around ~800×515	[11]
PTT release-day guide	Drag-box, double-click same type, Shift-add, Ctrl groups, right-click commands, skull timer, lower-right worker panel, tutorial sequence, building list, upgrade list	[6]

Evidence artifact	What it confirms	Source
GamesCheat review/ guide	Player building list; player unit HP/attack list; blacksmith upgrade list; continuous left/right expansion; renewable wood/food focus	12
StratBuster analysis	Start state; east/west dual-front structure; instantaneous unit-production criticism; Settler-only viability as a balance flaw	13
Hard/Easy walkthrough listings	Public longplay evidence exists, with hard run at 16:45 and easy run at 12:35	14
YouTube gameplay snippet	“Quick access clusters,” corroborating control-group-style unit grouping	15

The game world itself is best described as **continuous horizontal territory expansion**, not mission-to-mission discrete maps. The PTT guide explicitly says the game has no traditional “levels”; instead the player expands left and right, with monsters generally occupying resource zones and enemy strongholds sometimes holding captives that can be freed to join the fight. The same guide says successful assaults can trigger “siege battles,” after which the player rescues a princess and receives a reward. That aligns with the Armor/Newgrounds descriptions emphasizing princess rescue and fortress escalation. 16

Mechanics and system behaviors

The table below consolidates the principal mechanics/systems that are directly supported by the sources and translates each into a Pygame implementation approach.

System	Precise behavior observed or strongly implied	Player inputs	UI feedback	Edge cases to preserve or fix	Pygame implementation approach
World and camera	Continuous 2D side-scrolling world with hostile wilderness to the west and Vilereck’s territory to the east; expansion rather than discrete levels. 17	Mouse-driven scrolling or edge-scroll; optional keyboard scroll for modernization	HUD remains fixed; world objects move under camera offset	Camera snapping too hard during alerts; camera-to-worker jump from lower-right status panel	Store world coordinates in continuous space. Render at `screen_x = world_x - camera_x`. Keep a fixed HUD layer; camera clamps to world bounds.

System	Precise behavior observed or strongly implied	Player inputs	UI feedback	Edge cases to preserve or fix	Pygame implementation approach
Selection	Drag-box selection; double-click selects same-type units; Shift adds units; Ctrl+number assigns a control group; gameplay videos mention “quick access clusters.” [18]	Left-click select, left-drag box, Shift-click additive selection, Ctrl+number set group, number recall group	Highlight ring; selected portraits/ panel; selection box rectangle	Mixed unit/ building selection; dead or freed prisoner entering current selection; selection surviving destroyed object cleanly	Use `pygame.Rect` for box selection and `collidepoint` / `colliderect` for hit-tests. Maintain `selected_ids: set[int]` and `control_groups: dict[int, list[int]]`. [19]
Action issuing	Right-click initiates unit actions. Tutorial sequence confirms move, repair, gather, attack. [20]	Right-click point/entity	Cursor change or order decal; optional move marker and target ring	Invalid target type; target removed before arrival; command on inaccessible side of blocked footprint	Use contextual command resolution based on hovered entity class. Convert click to `Command` object and push to unit queue.

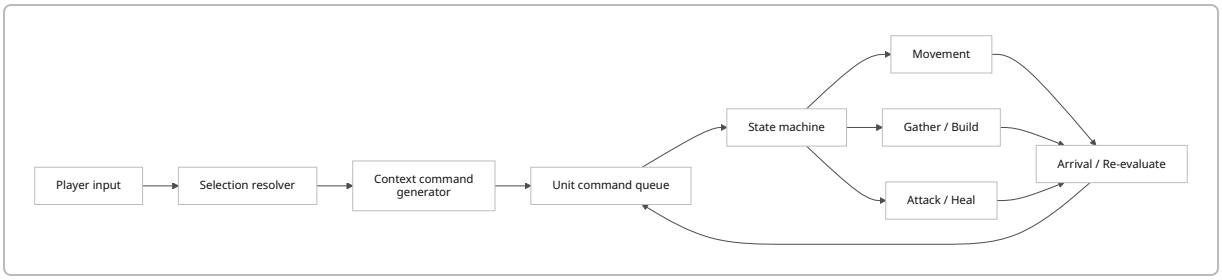
System	Precise behavior observed or strongly implied	Player inputs	UI feedback	Edge cases to preserve or fix	Pygame implementation approach
Worker economy	Settlers chop trees, mine stone/iron/gold, kill/harvest animals for food, deposit at huts, and build structures; wood and food are strategically renewable, while stone/iron/gold are spatially limited exploration resources. [21]	Select Settler, right-click resource or carcass; build menu for structures	Resource counters top-right; carry-state icon; worker-status panel lower-right	Resource depleted mid-swing; deposit destination destroyed; worker stuck holding cargo; simultaneous multiple workers on one node	Model workers with finite-state machines: `IDLE`, `MOVE`, `GATHER`, `RETURN`, `BUILD`, `ATTACK`. Static nodes expose remaining amount and gather type.
Building placement and construction	Tutorial explicitly requires building a Hut, later Barracks or Archery. Settlers build using gathered resources. Hut is both drop-off and population-growth node. [22]	Select Settler → Build button → choose building → confirm position	Placement footprint preview; blocked/valid tint; under-construction state	Invalid overlap with resources/buildings; not enough resources; moving builder killed during construction	Use static footprint rectangles for buildings. Reserve footprint on placement, then progress build percent during update loop.

System	Precise behavior observed or strongly implied	Player inputs	UI feedback	Edge cases to preserve or fix	Pygame implementation approach
Unit production	Hut produces Settlers and Spearmen; Barracks produces Spearman/ Swordman/ Mounted Swordsman; Archery produces Archers; Wizard Tower produces Wizards; Fortress recruits late-game heavy troops and siege. Some analyses say unit production is instantaneous, which may be a balance bug/ feature of the original. [23]	Select building → click unit button; optionally hold key or repeat for spam in modernization	Build panel; disabled buttons when unaffordable; population cap display	Original-style instant spawn can create runaway spam; spawn point blocked; building destroyed while queued	Implement per-building `production_queue`, but allow a faithful “zero-second” production mode behind data values for early replication.

System	Precise behavior observed or strongly implied	Player inputs	UI feedback	Edge cases to preserve or fix	Pygame implementation approach
Combat	Mixed melee and ranged combat. Settler is unusually flexible, functioning as worker and modest ranged/melee combatant. Wizard heals nearby allies. Catapult is slow, positional, and high-damage. Mounted units have special traits, including second-life behavior in community guides. [24]	Right-click attack target or move into aggro range	Health bars, hit flashes, projectile arcs, healing effects, death animation	Overkill target death before projectile lands; friendly unit body blocking siege; healer with no valid ally	Use per-unit `attack_cooldown`, `range`, `attack_type`, `projectile_type`, `heal_radius`, and separate impact resolution for melee/projectile/heal.
Defense structures	Watch Tower is the weakest defensive tower with one archer; Tower is stronger, with two archers. Iron resource page explicitly ties advanced defense to Tower and Fortress. [25]	Build placement, then passive auto-fire	Health bars, muzzle/projectile visuals, attack sound	Tower target priority; no valid target in range; tower under construction attacked	Treat towers as stationary units with ownership, attack loops, and multiple embedded-archer firing sockets if desired.

System	Precise behavior observed or strongly implied	Player inputs	UI feedback	Edge cases to preserve or fix	Pygame implementation approach
Waves, pressure, and objectives	A skull icon in the upper-right shows the next attack timing; western creatures periodically attack; enemy camps and castles can hold captives; rescuing princesses yields rewards; main goal is to reach and defeat Lord Vilereck in the east. [26]	Mostly indirect: prepare defenses, launch assaults, free captives	Wave timer, rescue popups, objective text	Captive freed during active combat; rescued units auto-selected unintentionally; boss death while local enemies remain	Use a scheduler for waves and event triggers, and `EncounterZone` objects for rescue/siege/boss transitions.
Upgrades	Blacksmith upgrades include spear, arrow, sword, infantry HP, bow range, catapult range, horse speed, mounted-knight mana, and wizard mana. [27]	Select Blacksmith and buy upgrade	Disabled/enabled upgrade buttons, icon glow, tooltip confirming tier	Duplicate purchase, upgrading dead-end tech path, upgrade affecting already-spawned units	Keep upgrades as faction-wide modifiers in player state, applied either dynamically or as stat caches refreshed on purchase.

A faithful-but-clean command model should be **queue-native** from the beginning, even if you initially expose only classic right-click behavior. That keeps the port close to the original while making modern improvements trivial later.



A workable command-object format for the port looks like this:

```
{
  "id": "cmd_1842",
  "type": "gather",
  "issued_at_ms": 128450,
  "issuer_player_id": 1,
  "target_entity_id": 720,
  "target_pos": [1840, 412],
  "queued": true,
  "metadata": {
    "resource_type": "wood",
    "deposit_building_id": 104
  }
}
```

For unit-state handling, a compact but sufficient model is:

```
{
  "entity_id": 32,
  "unit_type": "settler",
  "state": "RETURN",
  "hp": 40,
  "owner": "player",
  "world_pos": [1224.5, 409.0],
  "velocity": [48.0, 0.0],
  "selected": true,
  "carry": {
    "type": "wood",
    "amount": 8
  },
  "command_queue": [
    {"type": "deposit", "target_entity_id": 104},
    {"type": "gather", "target_entity_id": 721}
  ]
}
```

The most important pathing decision is the world representation. For this specific game, a **free/continuous 2D side-scroller with a walkable ground strip** is preferable to a full tile grid.

Approach	Fit to observed source material	Strengths	Weaknesses	Recommendation
Full tile grid with A* everywhere	Poor-to-moderate fit; the game is repeatedly described as a 2D side-scrolling plane with left/right territorial push rather than a dense tile-map RTS. [28]	Deterministic, easy blocked-footprint reasoning	More pathfinding overhead than the game seems to need; harder to preserve the loose Flash-era flow	Not recommended as the primary representation
Continuous world with static footprints and local steering	Best fit to side-view, left/right expansion, direct right-click action, and lane-like fronts. [29]	Simpler, more faithful feel, cheaper updates, easier high-FPS target	Requires careful anti-stuck logic when buildings cluster	Recommended default
Hybrid continuous world + coarse occupancy graph	Best for a polished port that starts faithful but adds QoL	Preserves original feel while solving detours around clustered buildings	Slightly more engineering than pure continuous motion	Recommended if the project will grow beyond a minimal clone

In practice, the port should use a **continuous x/y world**, with most meaningful motion occurring horizontally and only slight vertical offsets around the ground line, formation spread, and footprint detours. Static obstacles—buildings, mines, towers—can be recorded in a coarse occupancy structure, while units use local separation and “attempt straight-line else soft-detour” logic. Reserve true A* pathfinding only for rare reroutes around large player-made choke clusters; do not put every worker and soldier through full grid search every frame.

Rosters, stats, and asset inventory

Exact rosters from sources

The source-backed **player faction unit roster** is stable across the fandom caravan page and community guides: Settler, Spearman, Swordman, Mounted Swordsman, Archer, Wizard, Knight, Mounted Knight, and Catapult. The **resource roster** is exact and unambiguous: Wood, Stone, Iron, Gold, and Food. Release-era community guides list the buildable **player building roster** as Hut, Chicken Farm, Pig Farm, Barracks, Archery, Blacksmith, Wizard Tower, Watch Tower, Tower, and Fortress. Neutral animals are Deer, Pig, and Chicken. The wiki also confirms hostile Spider and Troll factions, and explicitly lists the Vilereck enemy

roster: Spearman (Vilereck), Archer (Vilereck), Swordman (Vilereck), Dark Knight, Dark Mounted Knight, Berzerk, and Lord Vilereck. [30]

Proposed faithful port stats for player units

Where exact values were recoverable from sources, they are kept. Where exact creation costs were not recoverable, the table proposes values that preserve source-described relative roles and tech order.

Unit	Source-backed role	HP	Damage	Speed px/s proposed	Range px proposed	Cost proposed	Population proposed	Notes
Settler	Worker; gathers all 5 resources; builds; modest ranged/ melee combatant [24]	40	6	78	115	20 Wood, 10 Food	1	Exact HP/ damage/ cost from wiki; keep weak but flexible. [31]
Spearman	Cheap early melee; weak versus arrows; common mass unit [32]	50	10	72	24	40 Wood, 20 Food	1	Exact HP/ damage/ cost from wiki. [33]
Swordman	Stronger core infantry than Spearman [12]	120	20	70	24	70 Wood, 20 Stone, 35 Food	1	HP/damage from 2013 guide; cost proposed from role. [12]
Mounted Swordsman	Fast cavalry; “50% chance of 2nd life” in guide [12]	175	30	108	28	95 Wood, 20 Stone, 45 Food	1	HP/damage exact from 2013 guide; retain revive mechanic as data-driven trait. [12]

Unit	Source-backed role	HP	Damage	Speed px/s proposed	Range px proposed	Cost proposed	Population proposed	Notes
Archer	Ranged support, weak in melee [34]	25	4	68	210	55 Wood, 15 Food	1	HP/damage from 2013 guide; Settler page corroborates Archer has 15 less HP than Settler. [34]
Wizard	Support healer; vulnerable in combat [12]	60	8	66	145	60 Wood, 15 Iron, 25 Gold, 40 Food	1	HP/damage from 2013 guide; treat damage as light magical projectile and healing aura/ability. [12]
Knight	Heavy all-round infantry; strongest standard infantry [35]	155	25	68	24	100 Wood, 15 Iron, 15 Gold, 75 Food	1	Exact HP/ damage/ cost from wiki. [35]
Mounted Knight	Elite cavalry; charge attack; “50% chance of 2nd life” [12]	200	35	112	30	120 Wood, 20 Iron, 30 Gold, 90 Food	1	HP/damage from 2013 guide; cost proposed as top cavalry. [12]

Unit	Source-backed role	HP	Damage	Speed px/s proposed	Range px proposed	Cost proposed	Population proposed	Notes
Catapult	Slow siege; high positional damage; must be protected [12]	30	75	42	300	80 Wood, 30 Stone, 25 Iron, 20 Gold, 30 Food	1	HP/damage from 2013 guide; original likely low survivability but strong siege burst. [12]

Proposed faithful port stats for player buildings

Building	Exact source-backed function	HP proposed	Build cost proposed	Build time proposed	Key behavior
Hut	Resource drop-off; increases population cap; produces Settlers and Spearmen. [22]	650	80 Wood	12 s	`dropoff=true`, `pop_cap_bonus=5`, `train=["settler","spearman"]`
Chicken Farm	Spawns chickens for food. [27]	240	50 Wood	8 s	Renewable low-rate food source
Pig Farm	Spawns pigs for food; stronger food engine than chickens in community strategy. [27]	300	60 Wood, 20 Stone	10 s	Renewable medium-rate food source

Building	Exact source-backed function	HP proposed	Build cost proposed	Build time proposed	Key behavior
Barracks	Produces Spearman, Swordman, Mounted Swordsman. [6]	700	100 Wood, 25 Stone	14 s	Core infantry/cavalry line
Archery	Produces Archers. [27]	560	90 Wood, 15 Stone	12 s	Ranged unit production
Blacksmith	Upgrades troop stats. [27]	600	120 Wood, 20 Stone, 10 Iron	14 s	Faction-wide upgrades
Wizard Tower	Produces Wizards. [27]	520	100 Wood, 15 Iron, 15 Gold	14 s	Healer/support production
Watch Tower	Weakest defense tower; one archer. [27]	700	90 Wood	12 s	Auto-attacks at long range
Tower	Stronger tower with two archers; iron page ties Tower to advanced resource use. [36]	1100	130 Wood, 40 Stone, 20 Iron	18 s	Durable defensive structure

Building	Exact source-backed function	HP proposed	Build cost proposed	Build time proposed	Key behavior
Fortress	Late-game military building; at minimum recruits Knights and Catapults, and in one guide is described as the source of all non-wizard plus elite units. [37]	1800	180 Wood, 60 Stone, 30 Iron, 25 Gold	24 s	Elite production, siege, possible late pop/command unlocks

Resource and upgrade data

Resource	Exact source-backed purpose	Renewable in practice	Port recommendation
Wood	Required for all units and most structures; gathered from trees by Settlers. [38]	Effectively yes, since trees are described strategically as the recurring backbone of play. [39]	Treat as abundant but geographically spread
Stone	Used for advanced structures and some units; mined by Settlers. [40]	No	Finite frontier expansion resource
Iron	Used for Tower, Fortress, and advanced units; mined by Settlers. [41]	No	Finite tech-gate resource
Gold	Used for expensive units and Fortress; mined by Settlers. [42]	No	Finite elite-resource gate
Food	Required for new units; gathered by slaying/harvesting Deer, Pig, or Chicken. [43]	Yes, via farms/animals	Renewable population-sustain resource

The Blacksmith upgrade tree is substantial enough that it should be data-driven from day one. Release-era guides list: **Sharp Spears / Mighty Spears**, **Sharp Arrow Tips / Mighty Arrow Tips**, **Sharp Swords / Mighty Swords**, **Boiled Leather / Fine Armors**, **Stief or Stiff Bows / War Bows**, **SiegeCraft**, **Fine Horseshoes**, **Glory**, and **Spell Mastery**. These affect spear damage, archer damage, swordsman damage, infantry HP, archer range, catapult range, horse speed, mounted-knight mana, and wizard mana. [27]

Estimated asset inventory for a faithful-but-original-art port

The quantities below assume a **mechanically faithful** port with **new original art**, not ripped assets. They are deliberately sized to match the observed complexity rather than to overproduce content.

Asset type	Estimated quantity	Frame estimate	Description
Player unit spritesheets	9 sheets	~240–290 total frames	Settler, Spearman, Swordman, Mounted Swordsman, Archer, Wizard, Knight, Mounted Knight, Catapult; typical loops: idle, walk, attack, death; plus gather/build for Settler, cast/heal for Wizard, charge/revive for cavalry
Enemy human spritesheets	7 sheets	~150–190 total frames	Vilereck Spearman, Archer, Swordman, Dark Knight, Dark Mounted Knight, Berzerk, Lord Vilereck
Monster spritesheets	4 sheets	~70–100 total frames	Spider, Alpha Spider, Troll Warrior, Troll ranged/ caster variant
Neutral animal sheets	3 sheets	~24–36 total frames	Deer, Pig, Chicken; include idle and death/harvest states
Building base sheets	10 sheets	Static or 2–4-frame ambient	Hut, Chicken Farm, Pig Farm, Barracks, Archery, Blacksmith, Wizard Tower, Watch Tower, Tower, Fortress
Enemy structure sheets	4–6 sheets	Mostly static	Tent, enemy outpost, enemy tower, cage, main castle, boss castle variant if desired
Construction/ damage state overlays	20–30 variants	Minimal	One under-construction pass plus one damaged pass for major buildings
Resource node textures	6–8	Static	Tree, stone deposit, iron mine, gold mine, carcass states, optional stump/depleted variants
Projectiles and VFX	10–16 sheets	~60–120 total frames	Arrows, rocks, heal pulse, charge effect, hit flashes, selection ring, move marker, damage numbers optional

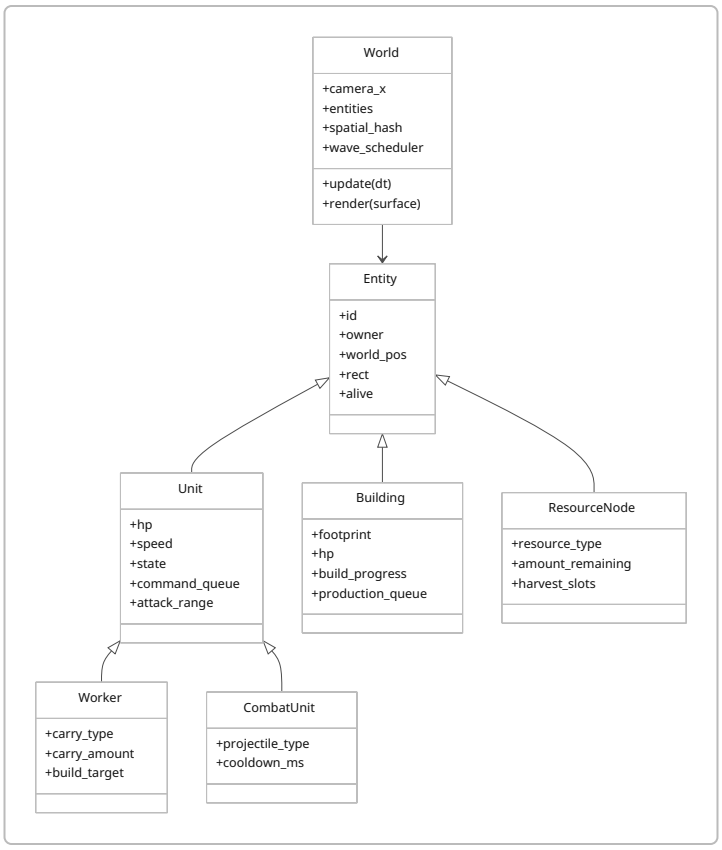
Asset type	Estimated quantity	Frame estimate	Description
Unit/building portrait icons	19–25	Static	9 player units, 10 player buildings, optional enemy portraits
Resource and upgrade icons	14–20	Static	5 resources, 9+ upgrades, pop cap, skull timer, build/repair icons
HUD/UI panels and widgets	35–55 pieces	Static	Top bar, lower command panel, tooltips, buttons, health bar parts, timers, selection box art, minimap frame optional
Cursor states	6–10	Static	Default, move, attack, gather, build-valid, build-invalid, repair
Sound effects	45–70	N/A	Clicks, confirms, invalid, build place, build complete, gather hits, deposits, melee hits, arrow shots, siege shots, healing, death, wave alerts
Voice barks	18–30	N/A	2–3 acknowledgements per player unit type, plus alert lines
Music loops	3–5	N/A	Calm economy, mid-pressure frontier, boss/siege, defeat, victory stinger optional

Those counts are rational because the source game's real content burden is not raw map diversity; it is **entity readability**. You need enough unit/building/UI art to make selection, ownership, production, wave timing, and frontline state legible at a glance. The biggest art time sink will be infantry/cavalry silhouettes and readable UI buttons, not backgrounds.

Pygame architecture and performance for 60–240 FPS

For this project, Pygame is not the limiting factor; architecture is. Pygame's sprite groups are explicitly designed to support efficient sprite membership management and allow the same sprite to exist in multiple organized groups, which is well aligned with RTS needs such as `render_group`, `selectable_group`, `enemy_group`, `worker_group`, and `projectile_group`. `Rect` objects give efficient hit-testing and rectangle overlap logic for selection and footprints, while `Clock.tick()` and `tick_busy_loop()` provide frame pacing choices. Pygame's image/surface documentation also explicitly recommends converting loaded images to a display-friendly format, using `convert()` or `convert_alpha()` as appropriate, which matters for blit-heavy RTS scenes. [4]

A solid entity model for the port looks like this:



For **60 FPS**, the target is straightforward. For **120 FPS**, it is still reasonable provided the simulation is not doing wasteful all-to-all checks. For **240 FPS**, the correct stance is “possible under moderate battle counts and careful coding,” not “guaranteed.” The right strategy is to make the simulation deterministic and efficient first, then allow rendering to run uncapped or capped at the display rate on stronger machines. Because Pygame’s timing resolution is platform-dependent and `tick_busy_loop()` trades CPU for tighter frame pacing, expose frame-capping as a user setting. [44]

A practical update/render split is:

```
handle_input()
resolve_selection_changes()
issue_commands()
fixed_simulation_step(dt_fixed)
  update_workers()
  update_units()
  update_projectiles()
  update_buildings()
  update_wave_scheduler()
  refresh_spatial_hash()
render()
  draw_background()
  draw_world_entities_sorted_by_y()
```

```
    draw_overlays()
    draw_hud()
```

The main optimization levers are clear:

Optimization area	Why it matters	Recommendation
Spatial partitioning	Avoid N×M range checks across all units/ projectiles	Use a uniform grid or spatial hash keyed by coarse world cells; only query nearby cells for aggro, target search, and projectile impacts
Surface conversion	Blit cost is meaningful in sprite-heavy scenes	`convert()` opaque art and `convert_alpha()` transparent art once at load time. [45]
Pre-scaling	Runtime scale transforms are expensive and visually inconsistent	Scale art at load time for the chosen virtual resolution; never rescale per frame
Update culling	Units far outside camera do not need full expensive thinking every frame	Keep off-screen animation cheap; full AI updates can run at a lower frequency if not near combat
Group organization	Selection, rendering, and collision should not traverse everything	Maintain specialized sprite/entity groups for render, selectable, hostile, worker, building, projectile. [46]
Sorting	Side-view overlap readability needs draw ordering	Sort renderables by ground `y` and secondary `x`, but cache order unless positions changed materially
Command validation	Dead/invalid targets create cascades of wasted logic	Invalidate queue entries lazily when target disappears; replan only then
Dirty-rect style updates	Historically useful, but less central on modern hardware	Prefer full-screen redraw for simplicity first; evaluate partial redraw only if profiling justifies it. Pygame documentation notes that older advice often emphasized dirty regions for performance. [47]

The original game also appears to have had optimization and robustness issues. PTT user comments explicitly complained that the game was not well optimized on older hardware, and the Settler wiki records a selection bug after a selected Hut was destroyed, requiring save/load recovery. Those are useful warnings: a faithful port should reproduce mechanics, not bugs. Selection invalidation, destroyed-building cleanup, and object-ID hygiene should be treated as mandatory engineering work, not polish. [22]

Networking is simple here: **do not build any** unless requirements change. The requested target is single-player desktop Pygame. That means no lockstep synchronization, no rollback, no prediction, and no

replication protocol. Instead, keep a single authoritative local simulation, deterministic where practical, with seedable RNG for reproducible test runs and save/load snapshots.

Engineering checklist, project structure, data schemas, and licensing

The checklist below is intentionally prioritized by dependency rather than by “phases.” Higher rows unblock more later systems.

System group	Core tasks	Complexity	Hours
Bootstrap and app shell	Windowing, virtual resolution policy, asset loader, scene loop, settings, save path setup	Easy	6–10
World and camera	Continuous world coordinates, edge/drag scroll, world bounds, parallax/background layers	Easy	8–14
Input and selection	Click select, box select, Shift-add, double-click same type, Ctrl groups, deselection rules	Medium	12–20
Command system	Right-click contextual orders, command objects, per-unit queues, cancel/replace semantics	Medium	14–24
Worker simulation	Gather → carry → deposit loop; build orders; repair orders; resource depletion	Hard	18–30
Resource economy	Resource nodes, renewable farm animals, top-bar counters, affordability checks	Medium	10–16
Building system	Footprint validation, placement preview, construction progress, destruction cleanup	Medium	14–24
Unit production	Building production panels, spawn points, queue handling, population-cap enforcement	Medium	12–22
Combat core	Aggro, melee/ranged attacks, projectiles, hit resolution, death/removal	Hard	20–34
Special abilities	Wizard heal, mounted charge/revive trait, catapult siege splash/range rules	Hard	12–24
Defensive structures	Watch Tower/Tower targeting, passive fire, attack priorities	Medium	8–14
Enemy pressure	Wave timer/skull icon, west-side raids, localized camp AI, boss/castle encounters	Hard	18–32
Rescue and objectives	Cages, freed units, princess reward events, Lord Vilereck defeat logic	Medium	8–16

System group	Core tasks	Complexity	Hours
Upgrade system	Blacksmith tech tree, faction-wide modifiers, tooltip/UI integration	Medium	10–18
HUD and command panel	Resource bar, selected-unit/building panel, tooltips, error feedback, status pings	Medium	16–28
Save/load	Serialize world, entities, queues, RNG seed, objectives, upgrades	Medium	8–14
Data authoring	JSON definitions for units/buildings/resources/ upgrades/waves	Medium	12–24
Profiling and optimization	Spatial hash, culling, allocation cleanup, hot-path profiling, bug hardening	Hard	16–30
Audio integration	SFX routing, voice bark cooldowns, music state switching, volume options	Easy	8–14

A realistic total is therefore about **212 to 394 hours** if every listed system is built thoughtfully, while a tighter but still strong implementation can sit closer to the **180–320 hour** executive estimate if UI/audio/ optimization ambition is controlled.

A project structure that will scale well is:

```
house_of_wolves_port/
  main.py
  settings.py
  game_app.py

  assets/
    art/
      units/
      buildings/
      resources/
      ui/
      fx/
    audio/
      sfx/
      voice/
      music/
    fonts/

  data/
    units.json
    buildings.json
    resources.json
    upgrades.json
```

```
  waves.json
  factions.json

src/
  core/
    loop.py
    camera.py
    input.py
    assets.py
    serialization.py
    profiler.py

  world/
    world.py
    terrain.py
    encounter_zones.py
    spatial_hash.py

  entities/
    base.py
    unit.py
    worker.py
    combat_unit.py
    building.py
    resource_node.py
    projectile.py
    captive.py

  systems/
    selection.py
    commands.py
    economy.py
    production.py
    combat.py
    upgrades.py
    ai.py
    waves.py
    objectives.py

  ui/
    hud.py
    command_panel.py
    tooltips.py
    cursors.py
    notifications.py

tests/
  test_commands.py
```

```
    test_economy.py
    test_combat.py
    test_save_load.py
```

Recommended JSON schemas should stay boring and explicit.

Sample unit schema

```
{
  "settler": {
    "display_name": "Settler",
    "role": ["worker", "light_ranged", "builder"],
    "stats": {
      "hp": 40,
      "damage": 6,
      "speed": 78,
      "attack_range": 115,
      "attack_cooldown_ms": 900
    },
    "cost": {
      "wood": 20,
      "stone": 0,
      "iron": 0,
      "gold": 0,
      "food": 10
    },
    "population": 1,
    "capabilities": {
      "can_gather": true,
      "can_build": true,
      "can_repair": true,
      "can_attack": true,
      "carry_capacity": 8
    },
    "animations": {
      "idle": 6,
      "walk": 8,
      "attack": 6,
      "gather": 6,
      "build": 6,
      "death": 6
    },
    "sounds": {
      "select": ["settler_select_1", "settler_select_2"],
      "move": ["settler_move_1", "settler_move_2"]
    }
```

```
  }
}
```

Sample building schema

```
{
  "hut": {
    "display_name": "Hut",
    "stats": {
      "hp": 650,
      "footprint_px": [140, 110],
      "build_time_ms": 12000
    },
    "cost": {
      "wood": 80,
      "stone": 0,
      "iron": 0,
      "gold": 0,
      "food": 0
    },
    "functions": {
      "dropoff": true,
      "population_cap_bonus": 5,
      "trainable_units": ["settler", "spearman"]
    },
    "animations": {
      "construction": 4,
      "idle": 1,
      "damaged": 2,
      "destroyed": 4
    }
  }
}
```

Sample resource schema

```
{
  "wood_tree": {
    "resource_type": "wood",
    "amount": 250,
    "gather_time_ms": 900,
    "harvest_slots": 2,
    "depleted_replacement": "tree_stump"
  },
  "gold_mine": {
```

```
    "resource_type": "gold",
    "amount": 400,
    "gather_time_ms": 1200,
    "harvest_slots": 3,
    "depleted_replacement": "gold_mine_empty"
  }
}
```

Sample command schema

```
{
  "type": "attack",
  "issuer_ids": [31, 32, 33],
  "target_entity_id": 904,
  "target_pos": [4870, 402],
  "queued": false,
  "stance_overrides": {
    "chase_radius": 240,
    "hold_after_kill": true
  }
}
```

On licensing and asset sourcing, the safest recommendation is straightforward: **create original art/audio** wherever practical, especially for anything that would otherwise closely imitate the original game's expressive look. If you supplement with third-party assets, prefer **CC0/public-domain** material first because CC0 is the "no rights reserved" route and OpenGameArt explicitly describes it as effectively public-domain style reuse with no requirement to ask, credit, or notify the creator. **CC BY 4.0** is also workable, but it requires attribution; Creative Commons' current guidance emphasizes crediting the author, source, and license. **CC BY-SA** is usable for some content, but it introduces share-alike obligations that you may not want in a portfolio project with mixed-source art. In practice, for a portfolio RTS port, the cleanest stack is: original gameplay code, original UI, original silhouettes, and either fully original art/audio or a tightly documented pool of CC0/CC BY assets with a dedicated credits file. [48]

The bottom-line recommendation is therefore:

1. Recreate the game as a **single-player, continuous, side-scrolling RTS** with **faithful roster, tutorial cadence, input model, and economy loop**.
2. Implement it on a **continuous-world command/state architecture**, not a heavy tile-grid RTS stack.
3. Keep all gameplay content **data-driven** from the start.
4. Aim for **faithful mechanics first**, then add modern quality-of-life features such as Shift queues, better control-group UX, improved AI, production timers, and sharper counters.
5. Use **original replacement assets** and explicit license hygiene so the project is portfolio-safe even if the mechanics are intentionally close to the 2013 source. [49]

---

1 5 9 House of Wolves - Play on Armor Games
https://armorgames.com/play/15014/house-of-wolves?utm_source=chatgpt.com

2 3 7 13 StratBuster - House of Wolves - StratBuster
https://stratbuster.weebly.com/house-of-wolves.html

4 46 pygame.sprite — pygame v2.6.0 documentation
https://www.pygame.org/docs/ref/sprite.html?utm_source=chatgpt.com

6 16 17 18 22 23 25 26 27 29 49 [分享] 策略-惡狼戰場（House of Wolves） - 看板 Little-Games - 批踢踢實業坊
https://www.ptt.cc/bbs/Little-Games/M.1368103154.A.BEC.html

8 14 HOUSE OF WOLVES free online game on Miniplay.com
https://www.miniplay.com/game/house-of-wolves

10 House of Wolves | The House of Wolves (RTS) Wiki | Fandom
https://the-house-of-wolves-rts.fandom.com/wiki/House_of_Wolves

11 20 House of Wolves Online Game | Gameflare.com
https://www.gameflare.com/online-game/house-of-wolves/

12 28 34 37 39 Games Cheat: May 2013
https://gamescheat22.blogspot.com/2013/05/

15 House of Wolves (Armor Games) Gameplay
https://www.youtube.com/watch?v=zvvI5JMFTeI&utm_source=chatgpt.com

19 pygame.Rect — pygame v2.6.0 documentation
https://www.pygame.org/docs/ref/rect.html?utm_source=chatgpt.com

21 Category:Resources | The House of Wolves (RTS) Wiki | Fandom
https://the-house-of-wolves-rts.fandom.com/wiki/Category%3AResources

24 31 Settler | The House of Wolves (RTS) Wiki - Fandom
https://the-house-of-wolves-rts.fandom.com/wiki/Settler?utm_source=chatgpt.com

30 House of Wolves (Caravan)
https://the-house-of-wolves-rts.fandom.com/wiki/House_of_Wolves_%28Caravan%29?utm_source=chatgpt.com

32 33 Spearman | The House of Wolves (RTS) Wiki | Fandom
https://the-house-of-wolves-rts.fandom.com/wiki/Spearman?utm_source=chatgpt.com

35 Knight | The House of Wolves (RTS) Wiki - Fandom
https://the-house-of-wolves-rts.fandom.com/wiki/Knight?utm_source=chatgpt.com

36 41 Iron | The House of Wolves (RTS) Wiki | Fandom
https://the-house-of-wolves-rts.fandom.com/wiki/Iron

38 Wood | The House of Wolves (RTS) Wiki | Fandom
https://the-house-of-wolves-rts.fandom.com/wiki/Wood

40 Stone | The House of Wolves (RTS) Wiki | Fandom
https://the-house-of-wolves-rts.fandom.com/wiki/Stone

42 Gold | The House of Wolves (RTS) Wiki | Fandom
https://the-house-of-wolves-rts.fandom.com/wiki/Gold

43 Food | The House of Wolves (RTS) Wiki | Fandom
https://the-house-of-wolves-rts.fandom.com/wiki/Food

44 pygame.time — pygame v2.6.0 documentation
https://www.pygame.org/docs/ref/time.html?utm_source=chatgpt.com

45 pygame.image — pygame v2.6.0 documentation
https://www.pygame.org/docs/ref/image.html?utm_source=chatgpt.com

47 A Newbie Guide to pygame
https://www.pygame.org/docs/tut/newbieguide.html?highlight=draw+rectangle%2F1000&utm_source=chatgpt.com

48 Public Domain
https://creativecommons.org/public-domain/?utm_source=chatgpt.com